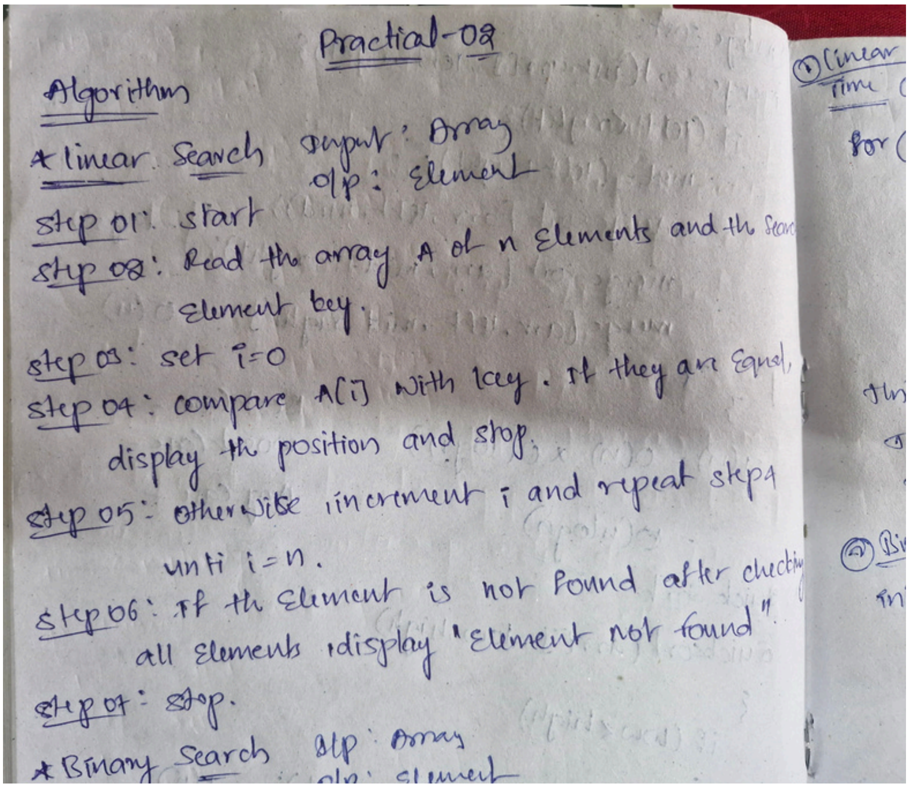
Practical: 2	Implementation and Time analysis of linear and binary search algorithm
05/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:



Practical-02

Algorithm

* linear Search input: Array
o/p: Element

step 01: start
step 02: Read the array A of n Elements and the search element key.
step 03: set i=0
step 04: compare A[i] with key. If they are Equal, display the position and stop.
step 05: otherwise increment i and repeat step4 until i=n.
step 06: If the Element is not found after checking all elements display "Element not found"
step 07: stop.

* Binary Search i/p: Array

Program:

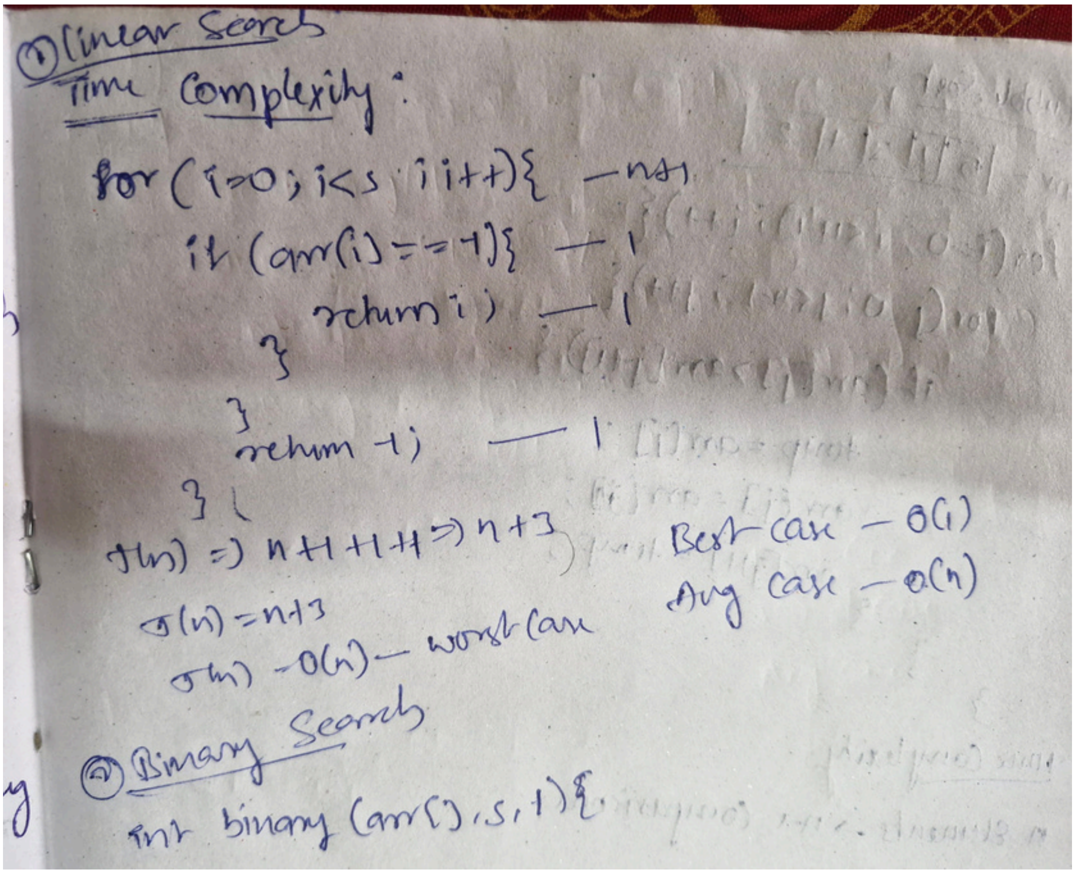
```
#include <iostream>
using namespace std;
int linearsearch(int arr[], int n, int target)
{
   for (int i = 0; i <= n - 1; i++)
   {
      if (arr[i] == target)
      {
         cout << "the target found at index of   " << i << "   elemnt is " << arr[i] << endl;
      }
   }
   return -1;
}
int main()
{
   int arr[] = {2, 3, 4, 5, 6, 7, 8, 9, 14};
   int n = sizeof(arr) / sizeof(arr[0]);
   int target = 14;
```

```
    linearsearch(arr, n, target);
    return 0;
}
```

Output:

```
Sorted array: 1 3 6 10
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ linearsearch.cpp -o linearsearch } ; if ($?) { .\linearsearch }
the target found at index of   8   elemnt is 14
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time Complexity:

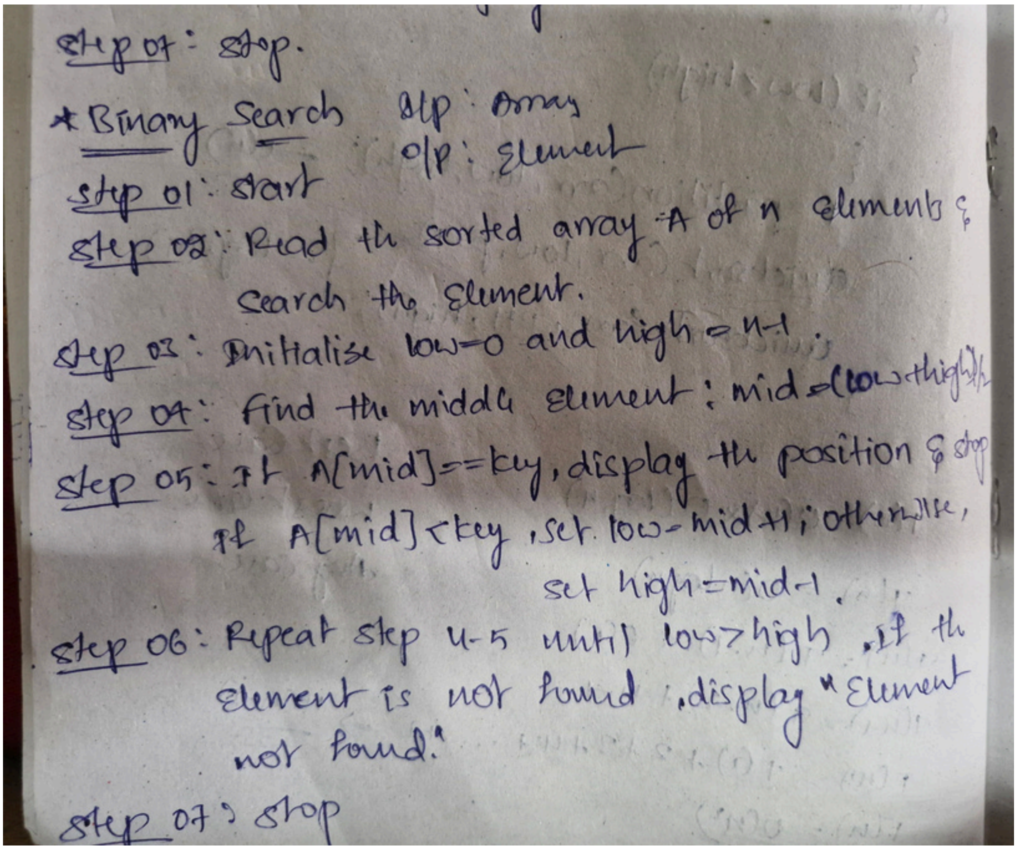

(1) linear Search
Time Complexity:
for (i=0; i<s; i++){ — n+1
if (arr(i)==t)){ — 1
return i; — 1
}
}
return -1; — 1
}
T(n) => n+1+1+1 => n+3
T(n) = n+3
T(n) - O(n) — worst case
Best case — O(1)
Avg case — O(n)
(2) Binary Search
int binary (arr[], s, t){

Conclusion:

Hence,From this practical of linear search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.

2.Binary search:

Algorithm:



step 07: stop.

* Binary Search i/p: Array
o/p: Element

step 01: start
step 02: Read the sorted array A of n elements & search the element.
step 03: Initialise low=0 and high = n-1.
step 04: Find the middle element: mid=(low+high)/2
step 05: If A[mid]==key, display the position & stop
If A[mid]<key, set low=mid+1; otherwise, set high=mid-1.
step 06: Repeat step 4-5 until low>high. If the element is not found, display "Element not found."
step 07: stop

Program:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n)
{
   for (int i = 0; i < n - 1; i++)
   {
      for(intj=0;j<n-i-1;j++)
      {
         if(arr[j] > arr[j+ 1])
         { // ascending order
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}
```

```
voidprintArray(int arr[], int n)
{
   for(inti=0; i < n; i++) {

   }  cout<<arr[i] << " ";
   cout<<endl;

}

int main()
{
   intarr[]={10, 3, 1, 45, 33, 9};
   intn=sizeof(arr) / sizeof(arr[0]);
   cout<<"Before sorting: ";
   printArray(arr, n);
   bubbleSort(arr, n);

   cout<<"After sorting: ";
   printArray(arr, n);
   return 0;

}
```

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Before sorting: 10 3 1 45 33 9
After sorting: 1 3 9 10 33 45
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time Complexity:

(a) Binary Search

```
int binary (arr[], s, t){
    l=0
    h=s-1;
    while (l<=h){
        mid=(l+h)/2;
        if (arr[m]==t) {
            return mid;
        }
        else if(arr[mid]<t){
            l=mid+1
        }
        else
        {   h=mid-1;
        }
    }
    return -1;
```

$n. \; n/2 + n/2 + n/2^k$

$n/2^k$

$\frac{n}{2^k} = 1 \Rightarrow n = 2^k$

$k = \log n$

Time complexity

$T(n) = O(\log n)$

Worst case – $O(\log n)$
Best Case – $O(1)$
Avg case – $O(\log n)$

Conclusion:

Hence,From this practical of Binary search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.